

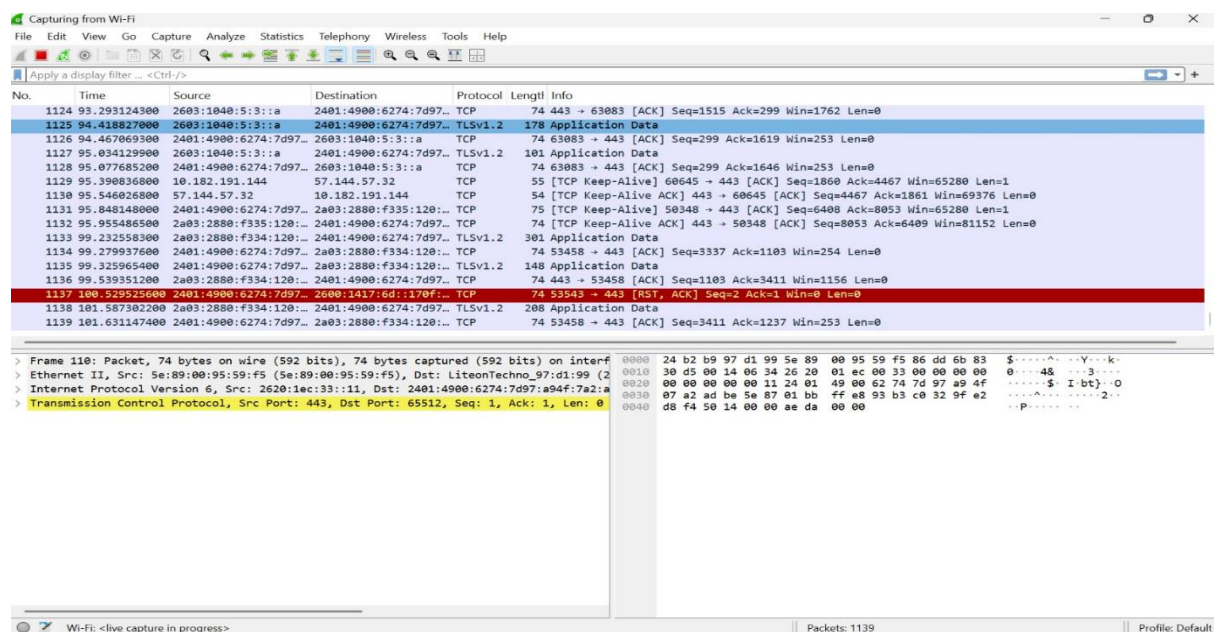
Password Capturing

Wireshark can capture not only passwords but any type of information transmitted over the network: usernames, email addresses, personal information, etc. As long as we can capture network traffic, Wireshark can sniff passing passwords.

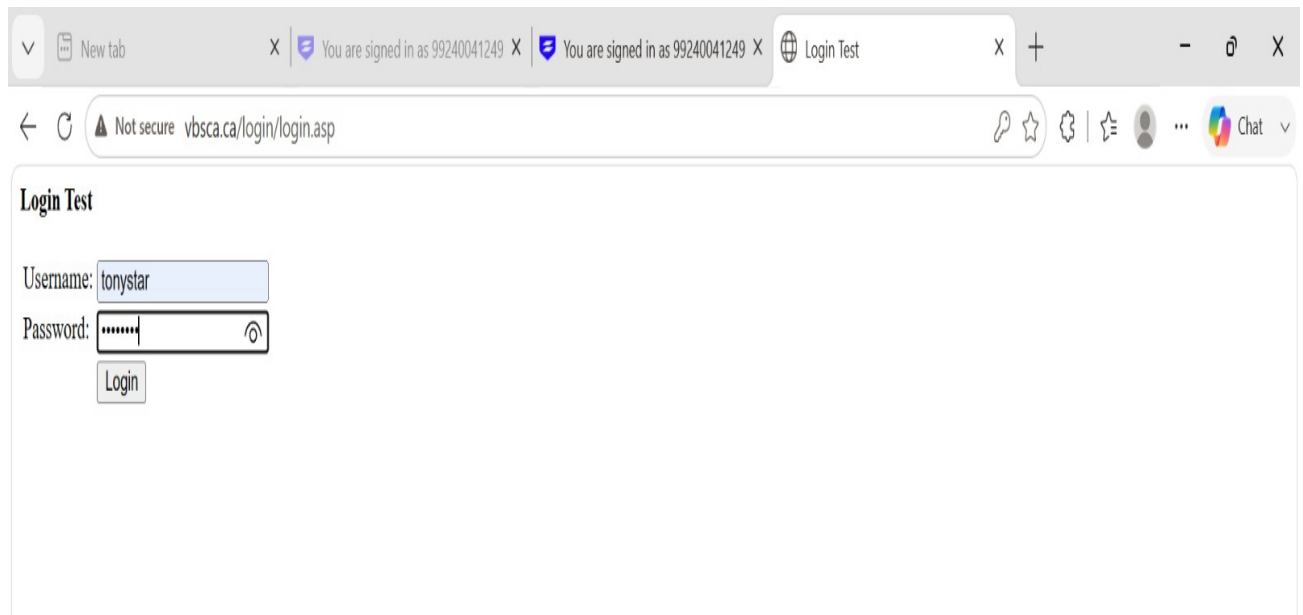
In sniffing can include passwords for various protocols such as HTTP, FTP, Telnet, etc. the captured data can be used to troubleshoot network problems, but can also be used maliciously to gain unauthorized access to sensitive information.

So, here we will see how we can capture the password using the Wireshark network capture analyzer. and see the outputs of the following steps.

Step 1: First of all, open your Wireshark tool in your window or in Linux virtual machine. and start capturing the network. suppose I am capturing my wireless fidelity.



Step 2: After starting the packet capturing we will go to the website and login the credential on that website as you can see in the image



Step 3: Now after completing the login credential we will go and capture the password in Wireshark. for that we have to use some filter that helps to find the login credential through the packet capturing.

Step 4: Wireshark has captured some packets but we specifically looking for HTTP packets. so in the display filter bar we use some command to find all the captured HTTP packets. as you can see in the below image the green bar where we apply the filter.

http

Wireshark packet capture showing HTTP traffic. The packet list pane shows several HTTP GET requests. The packet details pane for packet 51691 shows the Hypertext Transfer Protocol section with the request line: GET /login/login.asp HTTP/1.1. The packet bytes pane shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
20259	50.888341500	192.178.173.94	10.10.8.170	PKIX-C...	1107	Certificate Revocation List
25746	58.243075700	10.10.8.170	23.34.81.58	HTTP	165	GET /connecttest.txt HTTP/1.1
26479	58.959975300	23.34.81.58	10.10.8.170	HTTP	241	HTTP/1.1 200 OK (text/plain)
39082	89.992049000	10.10.8.170	23.34.81.48	HTTP	165	GET /connecttest.txt HTTP/1.1
39251	90.325084300	23.34.81.48	10.10.8.170	HTTP	241	HTTP/1.1 200 OK (text/plain)
51625	120.681741000	10.10.8.170	23.32.91.47	HTTP	165	GET /connecttest.txt HTTP/1.1
51691	120.901835600	10.10.8.170	23.155.129.172	HTTP	552	GET /login/login.asp HTTP/1.1
51720	121.002641400	23.32.91.47	10.10.8.170	HTTP	241	HTTP/1.1 200 OK (text/plain)
52096	121.626847900	23.155.129.172	10.10.8.170	HTTP	799	HTTP/1.1 200 OK (text/html)
52110	121.736166800	10.10.8.170	23.155.129.172	HTTP	510	GET /favicon.ico HTTP/1.1
52621	122.554123200	23.155.129.172	10.10.8.170	HTTP	1337	HTTP/1.1 404 Object Not Found (text/html)
57698	134.034836000	10.10.8.170	23.155.129.172	HTTP	785	POST /login/login_results.asp HTTP/1.1 (application/x-www-form-urlencoded)
57795	134.523175700	23.155.129.172	10.10.8.170	HTTP	143	HTTP/1.1 100 Continue
57920	134.870856100	23.155.129.172	10.10.8.170	HTTP	376	HTTP/1.1 200 OK (text/html)
64737	151.452240200	10.10.8.170	23.32.91.47	HTTP	165	GET /connecttest.txt HTTP/1.1
64855	151.664584200	23.32.91.47	10.10.8.170	HTTP	241	HTTP/1.1 200 OK (text/plain)

Frame 51691: Packet, 552 bytes on wire (4416 bits), 552 bytes captured (4416 bits) on Ethernet II, Src: LiteonTechno_97:d1:99 (24:b2:b9:97:d1:99), Dst: Sophos_fc:00:09 (c8:00:06:00:00:09), Internet Protocol Version 4, Src: 10.10.8.170, Dst: 23.155.129.172, Transmission Control Protocol, Src Port: 64484, Dst Port: 80, Seq: 1, Ack: 1, Len: 498, Hypertext Transfer Protocol

Hypertext Transfer Protocol (http), 498 bytes

Packets: 66448 - Displayed: 21 (0.0%)

Profile: Default

Step 5: So there are some HTTP packets are captured but we specifically looking for form data that the user submitted to the website. for that, we have a separate filter

As we know that there are main two methods used for submitting form data from web pages like login forms to the server. the methods are-

GET

POST

Step 6: So firstly for knowing the credential we use the first method and apply the filter for the GET methods as you can see below.

`http.request.method == "GET"`

GET method

As you can see in the image there are two packets where the login page was requested with a GET request as well, but

The image shows a Wireshark packet capture window. The top pane displays a list of network packets. Packet 51691 is highlighted, showing an HTTP GET request for /login/login.asp. The bottom pane shows the details of this packet, including the request method (GET), request URI (/login/login.asp), request version (HTTP/1.1), host (vbbsca.ca), connection (keep-alive), user-agent (Mozilla/5.0), and various headers like Accept, Accept-Encoding, and Accept-Language. The packet bytes pane shows the raw data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
26479	58.959975300	23.34.81.58	10.10.8.170	HTTP	241	HTTP/1.1 200 OK (text/plain)
39082	89.992049000	10.10.8.170	23.34.81.48	HTTP	165	GET /connecttest.txt HTTP/1.1
39251	90.325084300	23.34.81.48	10.10.8.170	HTTP	241	HTTP/1.1 200 OK (text/plain)
51625	120.681741000	10.10.8.170	23.32.91.47	HTTP	165	GET /connecttest.txt HTTP/1.1
51691	120.901835600	10.10.8.170	23.155.129.172	HTTP	552	GET /login/login.asp HTTP/1.1

Frame 51691: Packet, 552 bytes on wire (4416 bits), 552 bytes captured (4416 bits) on Ethernet II, Src: LiteonTechno_97:d1:99 (24:b2:b9:97:d1:99), Dst: Sophos_fc:00:09 (08:00:0c:00:09:00), Internet Protocol Version 4, Src: 10.10.8.170, Dst: 23.155.129.172, Transmission Control Protocol, Src Port: 64484, Dst Port: 80, Seq: 1, Ack: 1, Len: 552, Hypertext Transfer Protocol

GET /login/login.asp HTTP/1.1\r\nRequest Method: GET\r\nRequest URI: /login/login.asp\r\nRequest Version: HTTP/1.1\r\nHost: vbbsca.ca\r\nConnection: keep-alive\r\nUpgrade-Insecure-Requests: 1\r\nUser-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36\r\nAccept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8\r\nAccept-Encoding: gzip, deflate\r\nAccept-Language: en-US,en;q=0.9,en-IN;q=0.8\r\n\r\n

there is no form data submitted with a GET request.

Step 7: Now after checking the GET method if we didn't find the form data, then we will try the POST method for that we will apply the filter on Wireshark as you can see.

http.request.method == "POST"

The image shows a Wireshark packet capture window. The top pane displays a list of network packets. Packet 75953 is highlighted, showing an HTTP POST request for /login/login.asp. The bottom pane shows the details of this packet, including the request method (POST), request URI (/login/login.asp), request version (HTTP/1.1), host (vbbsca.ca), connection (keep-alive), user-agent (Mozilla/5.0), and various headers like Accept, Accept-Encoding, and Accept-Language. The packet bytes pane shows the raw data in hexadecimal and ASCII, including the form data.

No.	Time	Source	Destination	Protocol	Length	Info
71165	523.728497100	2600:1417:6e::170f::...	2401:4900:6274:7d97::...	HTTP	261	HTTP/1.1 200 OK (text/plain)
71550	593.914152300	2401:4900:6274:7d97::...	2600:1417:6e::170f::...	HTTP	222	GET /connecttest.txt HTTP/1.1
71552	593.914416300	10.182.191.144	23.205.118.154	HTTP	201	GET /connecttest.txt HTTP/1.1
71553	594.323023600	23.205.118.154	10.182.191.144	HTTP	241	HTTP/1.1 200 OK (text/plain)
71557	594.425450500	2600:1417:6e::170f::...	2401:4900:6274:7d97::...	HTTP	261	HTTP/1.1 200 OK (text/plain)
71763	629.038462500	10.182.191.144	23.205.118.154	HTTP	201	GET /connecttest.txt HTTP/1.1
71764	629.038517700	2401:4900:6274:7d97::...	2600:140f:2400::686::...	HTTP	222	GET /connecttest.txt HTTP/1.1
71766	629.513009600	2600:140f:2400::686::...	2401:4900:6274:7d97::...	HTTP	261	HTTP/1.1 200 OK (text/plain)
71768	629.513179000	23.205.118.154	10.182.191.144	HTTP	241	HTTP/1.1 200 OK (text/plain)
75452	731.997994400	10.182.191.144	23.155.129.172	HTTP	550	GET /login/login.asp HTTP/1.1
75487	732.855011600	23.155.129.172	10.182.191.144	HTTP	799	HTTP/1.1 200 OK (text/html)
75496	732.950079700	10.182.191.144	23.155.129.172	HTTP	510	GET /favicon.ico HTTP/1.1
75589	734.402407000	23.155.129.172	10.182.191.144	HTTP	57	HTTP/1.1 404 Object Not Found (text/html)
75953	752.861832900	10.182.191.144	23.155.129.172	HTTP	785	POST /login/login_results.asp HTTP/1.1 (application/x-www-form-urlencoded)
76049	753.353978100	23.155.129.172	10.182.191.144	HTTP	143	HTTP/1.1 100 Continue
76058	753.982406000	23.155.129.172	10.182.191.144	HTTP	376	HTTP/1.1 200 OK (text/html)

Frame 75953: Packet, 785 bytes on wire (6280 bits), 785 bytes captured (6280 bits) on Ethernet II, Src: LiteonTechno_97:d1:99 (24:b2:b9:97:d1:99), Dst: 5e:89:00:95:59:f5 (5 08:00:27:5e:89:00:95:59:f5), Internet Protocol Version 4, Src: 10.182.191.144, Dst: 23.155.129.172, Transmission Control Protocol, Src Port: 57374, Dst Port: 80, Seq: 1, Ack: 1, Len: 731, Hypertext Transfer Protocol

HTML Form URL Encoded: application/x-www-form-urlencoded

Form item: "txtUsername" = "tonystar"

Form item: "txtPassword" = "12345678"

As you can see we have a packet with form data click on the packet with user info and the application URL encoded. and click on the down-

HTML form URL Encoded where the login credential is found. login credential as it is the same that we filed on the website in step 2.

Form item: "uname" = "tonystar"

Form item: "pass" = "12345678"